

Lab 3 – Advanced Dart Practice Exercises

1. Objective

This lab helps students strengthen their understanding of advanced Dart features—including asynchronous programming, streams, microtasks, JSON handling, and factory constructors—by completing five hands-on exercises that can run directly on DartPad or in Android Studio / VS Code.

2. Requirements

Complete **five mini-projects** demonstrating advanced Dart concepts:

1. **Product Model & Repository** – build a data model and a repository with both Future and Stream behaviors.
2. **User Repository with JSON** – simulate fetching and parsing JSON data from an API.
3. **Async + Microtask Debugging** – illustrate event-loop and microtask order.
4. **Stream Transformation** – apply functional operations such as `map()` and `where()` to a stream.
5. **Factory Constructors & Cache** – demonstrate singleton and factory patterns.

All tasks must:

- Execute without error in DartPad or Flutter’s integrated Dart runtime.
- Include brief inline comments explaining logic and expected output.
- Print key results to the console.

3. Guided Steps

Exercise 1 – Product Model & Repository

Goal: Understand Futures and Streams.

1. Define Product { id, name, price }.
2. Implement ProductRepository with:
 - `Future<List<Product>> getAll()`
 - `Stream<Product> liveAdded()` for real-time updates.
3. Use `StreamController.broadcast()` to emit new items.
4. Print results to console.

Bài học : Hiểu được cơ chế hoạt động của Stream: cần gọi Controller lắng nghe sự kiện tương tự như websocket.

Exercise 2 – User Repository with JSON

Goal: Practice JSON serialization / deserialization.

1. Create User { name, email } and User.fromJson(Map) constructor.
2. Simulate JSON list from an API.
3. Use Future<List<User>> to return parsed data.
4. Display results with print().

Bài học : Hiểu được cách chuyển đổi Json sang Object Dart.

Exercise 3 – Async + Microtask Debugging

Goal: Differentiate microtask and event queues.

1. Write a snippet with scheduleMicrotask() and Future(() {...}).
2. Print execution order.
3. Explain why microtasks run before event callbacks.

Bài học: Dart luôn ưu tiên : Synchronous code => Microtask code => Event Queue

Exercise 4 – Stream Transformation

Goal: Use functional stream operators.

1. Create a stream of numbers 1–5.
2. Transform values to their squares using map().
3. Filter even numbers with where().
4. Listen and print each emitted value.

Bài học:

Exercise 5 – Factory Constructors & Cache

Goal: Show how factory constructors implement caching.

1. Create a Settings class with private constructor.
2. Add a factory Settings() that returns a singleton instance.

3. Verify two instances refer to the same object (`identical(a, b) → true`).

4. Expected Results

- Each exercise compiles and prints its output in order.
- Console shows proper stream events, microtask execution sequence, and singleton confirmation.
- Students understand how to handle asynchronous flows and object creation patterns in Dart.

5. Submission

- **Deliverable:** Single Dart file or zipped project containing five completed exercises.
- **Evaluation Criteria:**
 - Correct implementation and syntax (40%)
 - Code clarity and comments (20%)
 - Output accuracy (20%)
 - Use of `async` / stream features (20%)
- **Exercise 1:** Hiểu `Future`, `Stream`, `StreamController`, Repository Pattern.
- **Exercise 2:** Hiểu JSON parsing (`jsonDecode`, `fromJson`), mô phỏng API.
- **Exercise 3:** Hiểu Event Loop, Microtask Queue, Event Queue.
- **Exercise 4:** Hiểu Stream Pipeline (`map`, `where`), tư duy Functional Programming.
- **Exercise 5:** Hiểu Factory Constructor, Cache, Singleton Pattern.